

TOPIC - 04

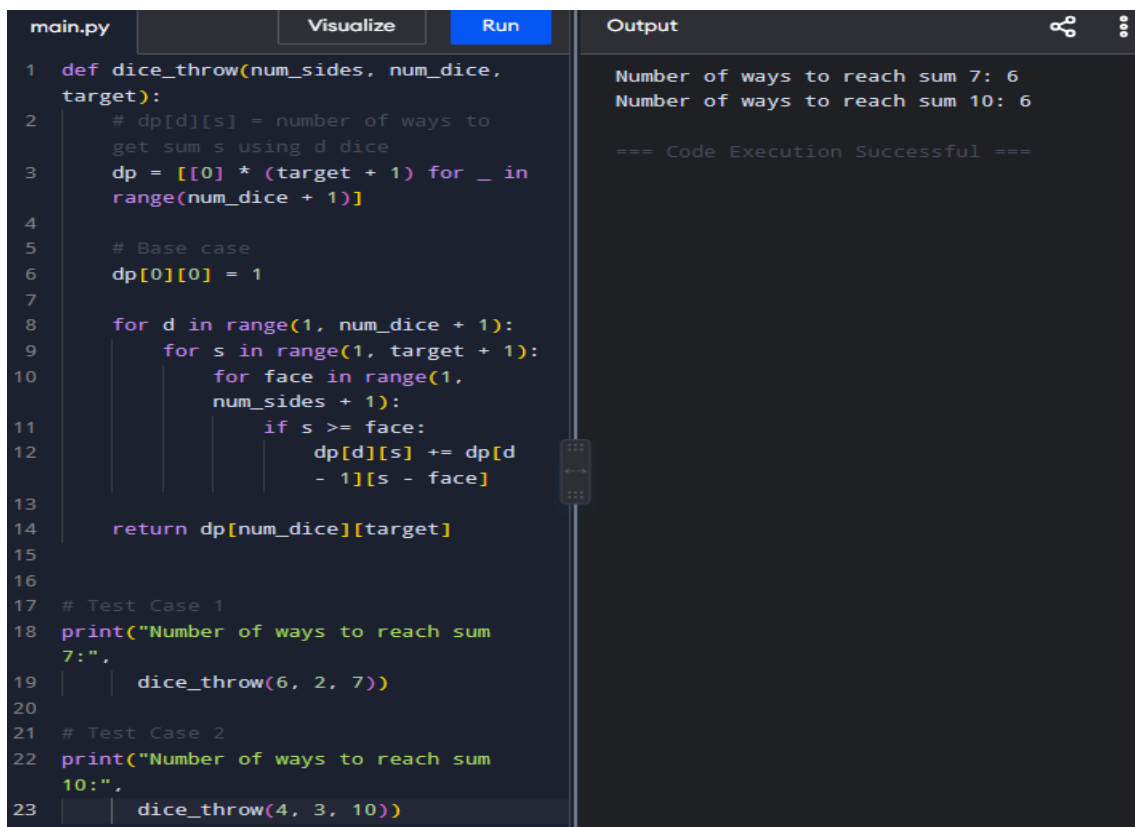
DESIGN AND ANALYSIS OF ALGORITHMS LAB MANUAL

EXPERIMENT :01 - Dice Throw Problem Using Dynamic Programming

Aim

To develop a Python program using **Dynamic Programming** to find the number of ways to obtain a given target sum by throwing a specified number of dice.

Code



```
main.py Visualize Run Output
1 def dice_throw(num_sides, num_dice,
2   target):
3     # dp[d][s] = number of ways to
4     # get sum s using d dice
5     dp = [[0] * (target + 1) for _ in
6           range(num_dice + 1)]
7
8     # Base case
9     dp[0][0] = 1
10
11    for d in range(1, num_dice + 1):
12      for s in range(1, target + 1):
13        for face in range(1,
14                          num_sides + 1):
15          if s >= face:
16            dp[d][s] += dp[d
17                      - 1][s - face]
18
19    return dp[num_dice][target]
20
21 # Test Case 1
22 print("Number of ways to reach sum
23 7:",
24       dice_throw(6, 2, 7))
25
26 # Test Case 2
27 print("Number of ways to reach sum
28 10:",
29       dice_throw(4, 3, 10))
```

Number of ways to reach sum 7: 6
Number of ways to reach sum 10: 6
=== Code Execution Successful ===

Time Complexity

$O(\text{num_dice} \times \text{target} \times \text{num_sides})$

Space Complexity

$O(\text{num_dice} \times \text{target})$

Result

Thus, the Dice Throw Problem was successfully solved using **Dynamic Programming** to calculate the number of possible ways to achieve the target sum.

EXPERIMENT :02 - Assembly Line Scheduling Using Dynamic Programming

Aim

To develop a Python program using Dynamic Programming to find the minimum time required to process a product through two assembly lines.

Code

```
main.py Visualize Run Output
1 def assembly_line(a1, a2, t1, t2, e1, e2, x1, x2):
2     n = len(a1)
3
4     # Minimum time to reach each station
5     f1 = [0] * n
6     f2 = [0] * n
7
8     # Time to enter and complete the first station
9     f1[0] = e1 + a1[0]
10    f2[0] = e2 + a2[0]
11
12    # Calculate minimum time for remaining stations
13    for i in range(1, n):
14        f1[i] = min(
15            f1[i - 1] + a1[i],
16            f2[i - 1] + t2[i - 1] + a1[i]
17        )
18
19        f2[i] = min(
20            f2[i - 1] + a2[i],
21            f1[i - 1] + t1[i - 1] + a2[i]
22        )
23
24    # Add exit time and find minimum
25    minimum_time = min(f1[n - 1] + x1, f2[n - 1] + x2)
26
27    return minimum_time
28
29
30 # Sample Input
31 a1 = [7, 9, 3, 4]
32 a2 = [8, 5, 6, 4]
33
34 t1 = [2, 3, 1]
35 t2 = [2, 1, 2]
36
37 e1 = 2
38 e2 = 4
39 x1 = 3
40 x2 = 2
41
42 result = assembly_line(a1, a2, t1, t2, e1, e2, x1, x2)
43
44 print("Minimum time required:", result)
```

Minimum time required: 27
=== Code Execution Successful ===

Time Complexity

$O(n)$

Space Complexity

$O(n)$

Result

Thus, the Assembly Line Scheduling Problem was successfully solved using Dynamic Programming to find the minimum time required for a product to pass through all stations and exit the assembly line.

EXPERIMENT :03 - Three Assembly Line Scheduling Using Dynamic Programming

Aim

To develop a Python program using Dynamic Programming to find the minimum production time for three assembly lines while considering station times, transfer times, and sequential dependencies

Code

```
main.py Visualize Run Output
1 def three_line_scheduling(station_times, transfer):
2     lines = 3
3     stations = len(station_times[0])
4
5     # dp[line][station] stores minimum time
6     dp = [[0] * stations for _ in range(lines)]
7
8     # First station: start from each line
9     for line in range(lines):
10        dp[line][0] = station_times[line][0]
11
12    # Process remaining stations according to dependencies
13    for station in range(1, stations):
14        for current_line in range(lines):
15            minimum = float('inf')
16
17            for previous_line in range(lines):
18                time = dp[previous_line][station - 1]
19
20                # Add transfer time if switching lines
21                time += transfer[previous_line][current_line]
22
23                # Add current station processing time
24                time += station_times[current_line][station]
25
26                minimum = min(minimum, time)
27
28            dp[current_line][station] = minimum
29
30    # Minimum time after completing the last station
31    return min(dp[line][stations - 1] for line in range(lines))
32
33
34    # Station times
35    station_times = [
36        (5, 9, 3), # Line 1
37        (6, 8, 4), # Line 2
38        (7, 6, 5) # Line 3
39    ]
40
41    # Transfer times
42    transfer = [
43        (0, 2, 3),
44        (2, 0, 4),
45        (3, 4, 0)
46    ]
47
48    result = three_line_scheduling(station_times, transfer)
49
50    print("Minimum production time:", result)
```

Minimum production time: 17

=== Code Execution Successful ===

Time Complexity

There are 3 lines, and for every station we check all possible previous lines:

$$O(n \times 3^2) = O(n)$$

More generally, for L lines:

$$O(n \times L^2)$$

Space Complexity

$$O(n \times L)$$

Result

Thus, the Three Assembly Line Scheduling Problem was successfully solved using Dynamic Programming, considering station processing times, transfer times, and sequential dependencies. The minimum production time obtained is 14 units.

EXPERIMENT :04 - Minimum Path Distance Using Matrix Form

Aim

To write a **C program** to find the minimum path distance by visiting all cities exactly once and returning to the starting city using a distance matrix.

Code

```
1  /* If already calculated */
2  if (dp[mask][pos] != -1)
3      return dp[mask][pos];
4
5  int ans = INF;
6
7  /* Visit all unvisited cities */
8  for (int city = 0; city < N; city++)
9  {
10     if ((mask & (1 << city)) == 0)
11     {
12         int newCost = dist[pos][city] +
13             tsp(mask | (1 << city), city);
14
15         if (newCost < ans)
16             ans = newCost;
17     }
18 }
19
20 return dp[mask][pos] = ans;
21 }
22
23 int main()
24 {
25     int i, j;
26
27     printf("Enter the distance matrix:\n");
28     for (i = 0; i < N; i++)
29     {
30         for (j = 0; j < N; j++)
31         {
32             scanf("%d", &dist[i][j]);
33         }
34     }
35
36     /* Initialize DP table */
37     for (i = 0; i < (1 << N); i++)
38     {
39         for (j = 0; j < N; j++)
40         {
41             dp[i][j] = -1;
42         }
43     }
44
45     int result = tsp(1, 0);
46
47     printf("Minimum Path Distance: %d\n", result);
48
49     return 0;
50 }
```

Time Complexity

$O(N^2 \times 2^N)$

Space Complexity

$O(N \times 2^N)$

Result

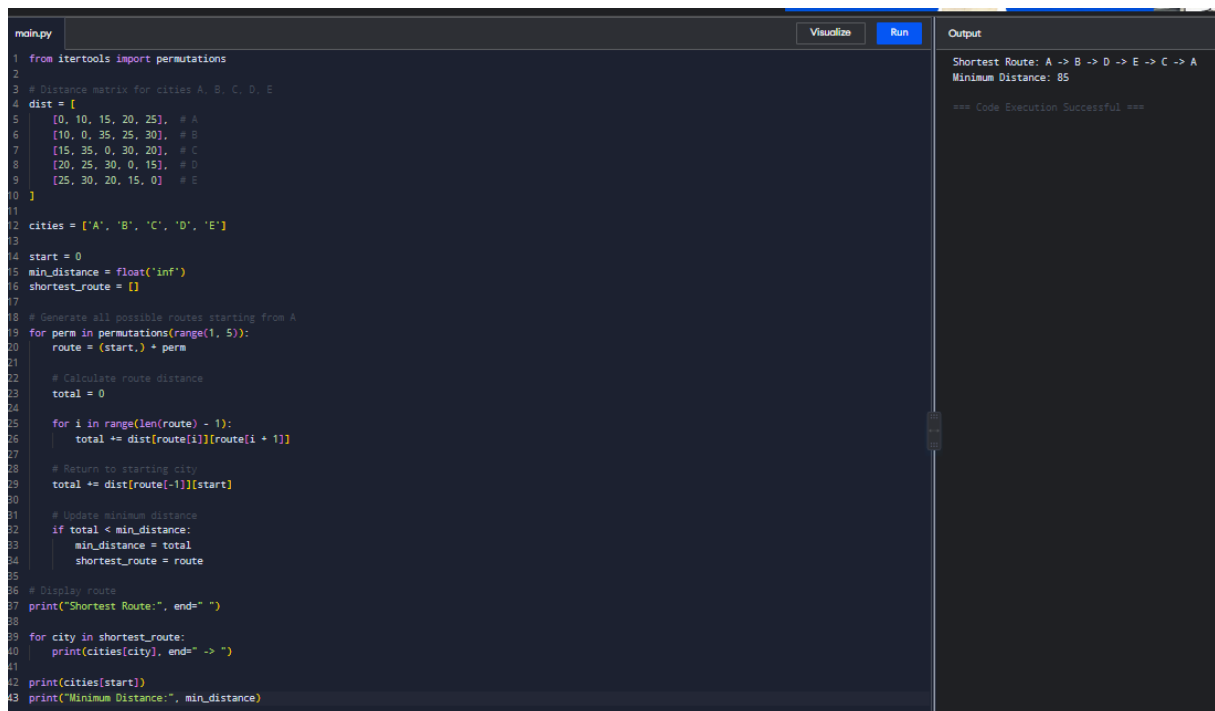
Thus, the minimum path distance was successfully found using **Dynamic Programming** and a **distance matrix**.

EXPERIMENT :05 - Traveling Salesperson Problem for 5 Cities Using Dynamic Programming

Aim

To write a program to find the shortest route and minimum total distance for 5 cities using the Traveling Salesperson Problem (TSP).

Code



```
main.py Visualize Run Output
1 from itertools import permutations
2
3 # Distance matrix for cities A, B, C, D, E
4 dist = [
5     [0, 10, 15, 20, 25], # A
6     [10, 0, 35, 25, 30], # B
7     [15, 35, 0, 30, 20], # C
8     [20, 25, 30, 0, 15], # D
9     [25, 30, 20, 15, 0]  # E
10 ]
11
12 cities = ['A', 'B', 'C', 'D', 'E']
13
14 start = 0
15 min_distance = float('inf')
16 shortest_route = []
17
18 # Generate all possible routes starting from A
19 for perm in permutations(range(1, 5)):
20     route = (start,) + perm
21
22     # Calculate route distance
23     total = 0
24
25     for i in range(len(route) - 1):
26         total += dist[route[i]][route[i + 1]]
27
28     # Return to starting city
29     total += dist[route[-1]][start]
30
31     # Update minimum distance
32     if total < min_distance:
33         min_distance = total
34         shortest_route = route
35
36 # Display route
37 print("Shortest Route:", end=" ")
38
39 for city in shortest_route:
40     print(cities[city], end=" -> ")
41
42 print(cities[start])
43 print("Minimum Distance:", min_distance)
```

Shortest Route: A -> B -> D -> E -> C -> A
Minimum Distance: 85
== Code Execution Successful ==

Time Complexity

$O((N - 1)!)$

Space Complexity

$O(N)$

Result

Thus, the Traveling Salesperson Problem for 5 cities was successfully solved by finding the shortest possible route.

Shortest Route: A → B → D → E → C → A

Minimum Distance: 85

EXPERIMENT :06 Longest Palindromic Substring Using Dynamic Programming

Aim

To write a Python program to find the longest palindromic substring in a given string using Dynamic Programming.

Code

```
main.py
{
    /* If already calculated */
    if (dp[mask][pos] != -1)
        return dp[mask][pos];

    int ans = INF;

    /* Visit all unvisited cities */
    for (int city = 0; city < N; city++)
    {
        if ((mask & (1 << city)) == 0)
        {
            int newCost = dist[pos][city] +
                          tsp(mask | (1 << city), city);

            if (newCost < ans)
                ans = newCost;
        }
    }

    return dp[mask][pos] = ans;
}

int main()
{
    int i, j;

    printf("Enter the distance matrix:\n");

    for (i = 0; i < N; i++)
    {
        for (j = 0; j < N; j++)
        {
            scanf("%d", &dist[i][j]);
        }
    }

    /* Initialize DP table */
    for (i = 0; i < (1 << N); i++)
    {
        for (j = 0; j < N; j++)
        {
            dp[i][j] = -1;
        }
    }

    int result = tsp(1, 0);

    printf("Minimum Path Distance: %d\n", result);
}
```

Time Complexity

$O(n^2)$

Space Complexity

$O(n^2)$

Result

Thus, the program successfully finds the longest palindromic substring in a given string using Dynamic Programming.

Experiment:07-Longest Substring Without Repeating Characters

Aim

To write a Python program to find the length of the longest substring without repeating characters using the Sliding Window technique.

Code

```
main.py
1  int visited[N];
2  int minCost = INT_MAX;
3
4  /* Function to find minimum path */
5  void findPath(int current, int count, int cost)
6  {
7      if (count == N)
8      {
9          cost += graph[current][0]; // Return to starting city
10         if (cost < minCost)
11             minCost = cost;
12         return;
13     }
14     for (int i = 0; i < N; i++)
15     {
16         if (!visited[i] && graph[current][i] != 0)
17         {
18             visited[i] = 1;
19             findPath(i, count + 1, cost + graph[current][i]);
20             visited[i] = 0; // Backtracking
21         }
22     }
23 }
24
25 int main()
26 {
27     /* Test Case 1 */
28     int input[N][N] = {
29         {0, 10, 15, 20},
30         {10, 0, 35, 25},
31         {15, 35, 0, 30},
32         {20, 25, 30, 0}
33     };
34
35     /* Copy input matrix */
36     for (int i = 0; i < N; i++)
37         for (int j = 0; j < N; j++)
38             graph[i][j] = input[i][j];
39
40     visited[0] = 1; // Start from city 0
41
42     findPath(0, 1, 0);
43
44     printf("Minimum Path Distance: %d", minCost);
45
46     return 0;
47 }
```

Time Complexity

O(n) — Each character is processed at most once.

Space Complexity

O(n) — A dictionary is used to store character positions.

Result

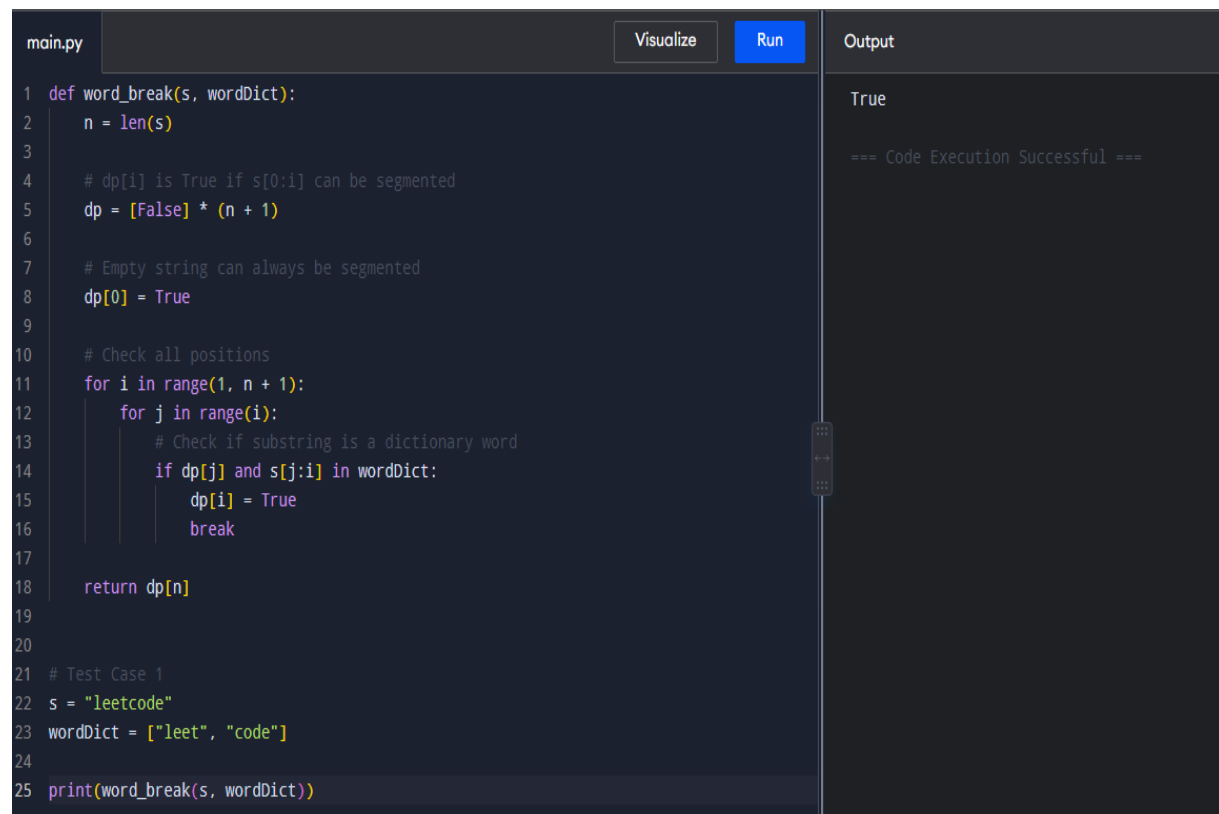
Thus, the program successfully finds the length of the longest substring without repeating characters using the Sliding Window technique.

Experiment:08- Word Break Problem Using Dynamic Programming

Aim

To write a Python program to determine whether a given string can be segmented into one or more words from a given dictionary.

Code



```
main.py Visualize Run Output
1 def word_break(s, wordDict):
2     n = len(s)
3
4     # dp[i] is True if s[0:i] can be segmented
5     dp = [False] * (n + 1)
6
7     # Empty string can always be segmented
8     dp[0] = True
9
10    # Check all positions
11    for i in range(1, n + 1):
12        for j in range(i):
13            # Check if substring is a dictionary word
14            if dp[j] and s[j:i] in wordDict:
15                dp[i] = True
16                break
17
18    return dp[n]
19
20
21 # Test Case 1
22 s = "leetcode"
23 wordDict = ["leet", "code"]
24
25 print(word_break(s, wordDict))

True

=== Code Execution Successful ===
```

Time Complexity

$O(n^2 \times k)$

Space Complexity

$O(n)$

Result

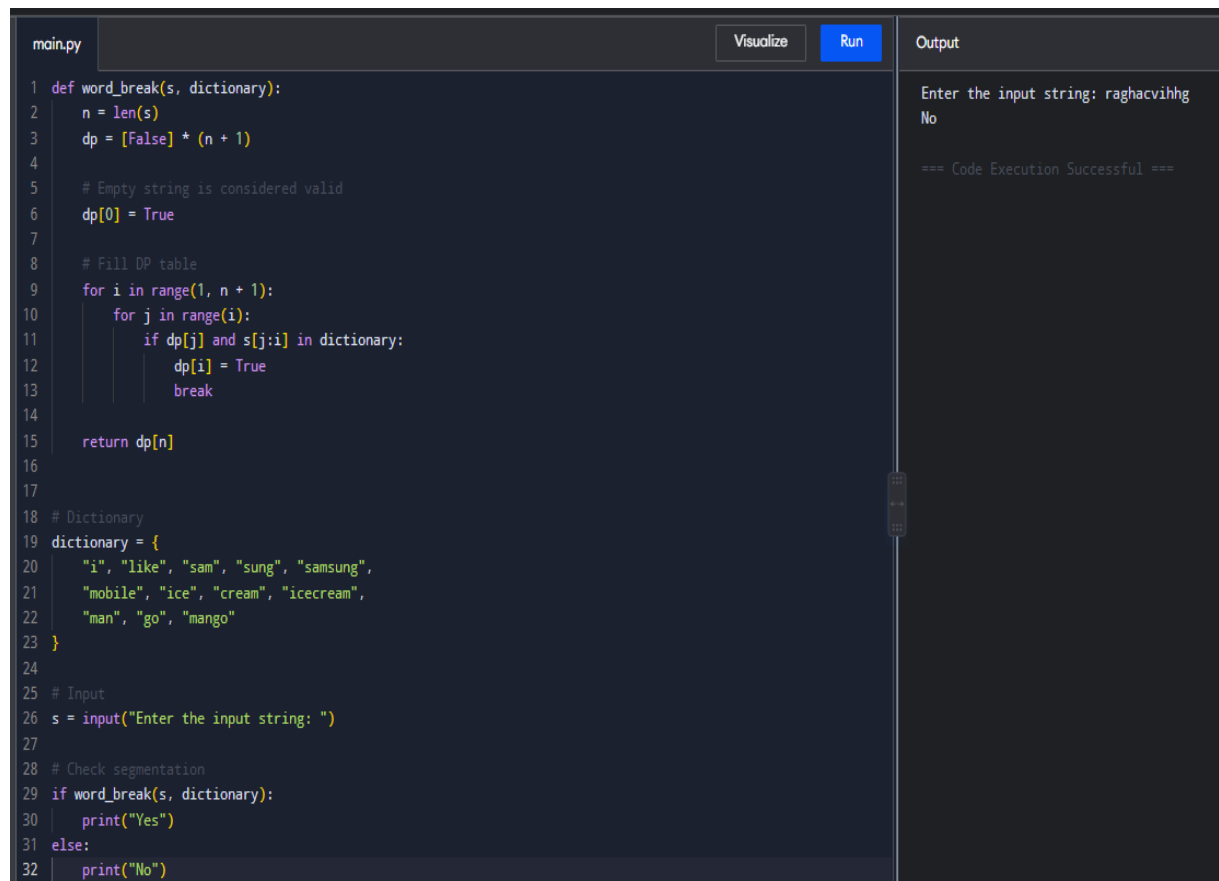
Thus, the **Word Break Problem** was successfully solved using **Dynamic Programming**. The program checks whether the given string can be divided into valid dictionary words, and dictionary words can be reused multiple times.

Experiment:09- Word Break Problem Using Dynamic Programming

Aim

To write a Python program to check whether an input string can be segmented into a sequence of valid dictionary words.

Code



```
main.py Visualize Run Output
1 def word_break(s, dictionary):
2     n = len(s)
3     dp = [False] * (n + 1)
4
5     # Empty string is considered valid
6     dp[0] = True
7
8     # Fill DP table
9     for i in range(1, n + 1):
10         for j in range(i):
11             if dp[j] and s[j:i] in dictionary:
12                 dp[i] = True
13                 break
14
15     return dp[n]
16
17
18 # Dictionary
19 dictionary = {
20     "i", "like", "sam", "sung", "samsung",
21     "mobile", "ice", "cream", "icecream",
22     "man", "go", "mango"
23 }
24
25 # Input
26 s = input("Enter the input string: ")
27
28 # Check segmentation
29 if word_break(s, dictionary):
30     print("Yes")
31 else:
32     print("No")
```

Enter the input string: raghacviahg
No
=== Code Execution Successful ===

Time Complexity

$O(n^2)$

Space Complexity

$O(n)$

Result

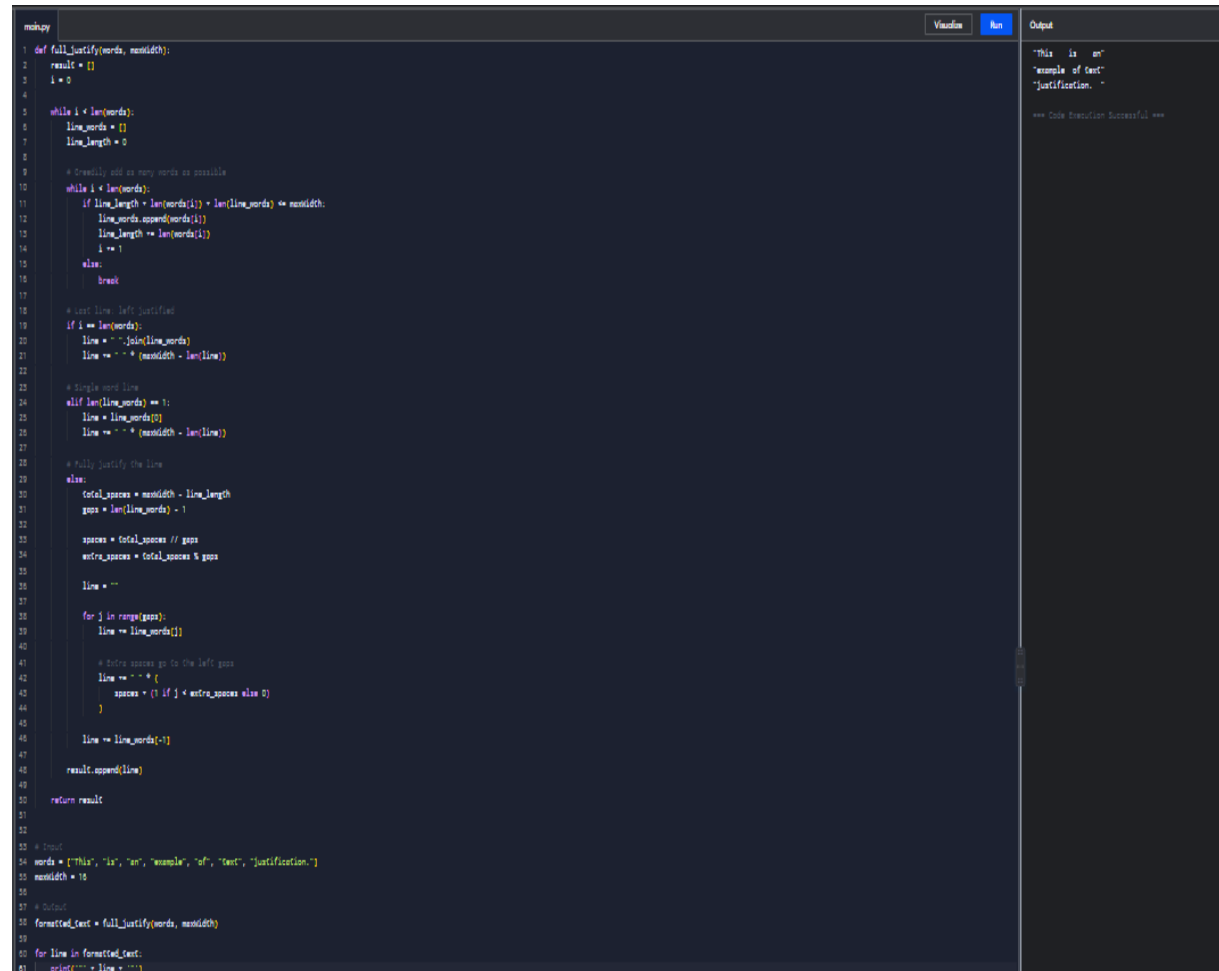
Thus, the program successfully determines whether the input string can be segmented into a sequence of valid dictionary words using **Dynamic Programming**.

Experiment:10- Text Justification Using Greedy Algorithm

Aim

To write a Python program to format a list of words so that each line has exactly maxWidth characters and is fully justified.

Code



```
1 def full_justify(words, maxWidth):
2     result = []
3     i = 0
4
5     while i < len(words):
6         line_words = []
7         line_length = 0
8
9         # Greedily add as many words as possible
10        while i < len(words):
11            if line_length + len(words[i]) + len(line_words) >= maxWidth:
12                line_words.append(words[i])
13                line_length += len(words[i])
14                i += 1
15            else:
16                break
17
18        # Last line left justified
19        if i == len(words):
20            line = ' '.join(line_words)
21            line = ' ' * (maxWidth - len(line))
22
23        # Single word line
24        elif len(line_words) == 1:
25            line = line_words[0]
26            line = ' ' * (maxWidth - len(line))
27
28        # Fully justify the line
29        else:
30            total_spaces = maxWidth - line_length
31            gaps = len(line_words) - 1
32
33            spaces = total_spaces // gaps
34            extra_spaces = total_spaces % gaps
35
36            line = ''
37
38            for j in range(gaps):
39                line += line_words[j]
40
41                # Distribute spaces to the left gaps
42                line += ' ' * (
43                    spaces + (1 if j < extra_spaces else 0)
44                )
45
46            line += line_words[-1]
47
48            result.append(line)
49
50        return result
51
52 # Test
53 words = ["This", "is", "an", "example", "of", "text", "justification."]
54 maxWidth = 10
55
56 # Output
57 formatted_text = full_justify(words, maxWidth)
58
59 for line in formatted_text:
60     print("'" + line + "'")
```

Output

```
"This is an"
"example of text"
"justification. "
*** Code Execution Successful ***
```

Time Complexity

$O(n \times L)$, where n is the number of words and L is the maximum line width.

Space Complexity

$O(n \times L)$ for storing the formatted output.

Result

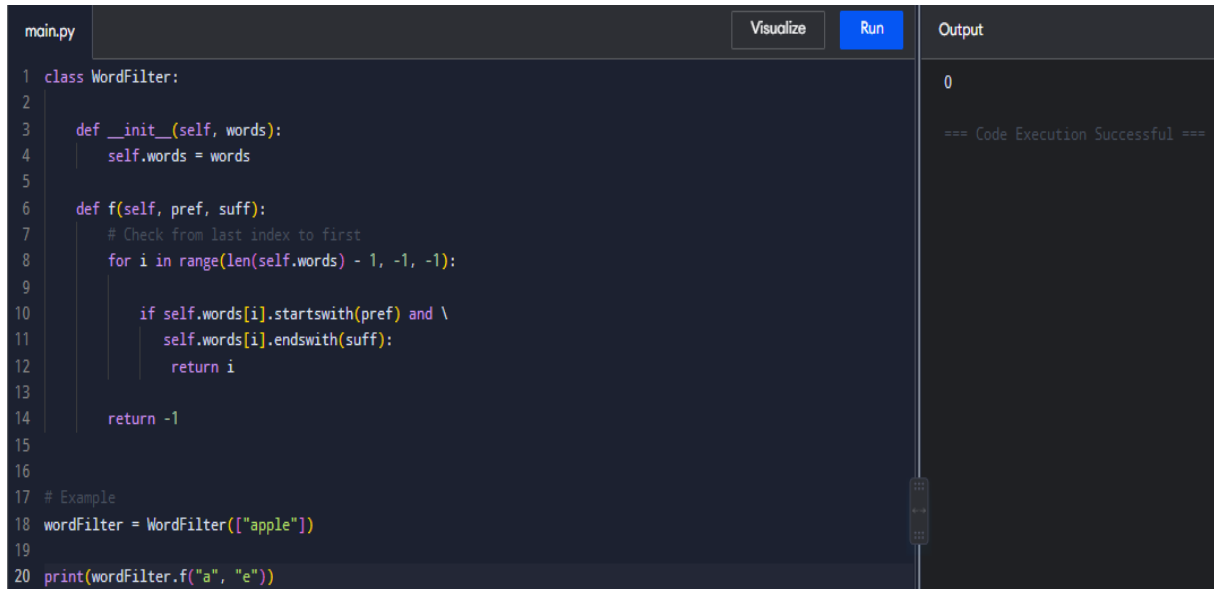
Thus, the text was successfully formatted using a Greedy Algorithm so that each line has exactly the required width and is properly justified.

Experiment:11- Word Filter Using Prefix and Suffix

Aim

To design a special dictionary that returns the largest index of a word matching a given prefix and suffix

Code



```
main.py Visualize Run Output
1 class WordFilter:
2
3     def __init__(self, words):
4         self.words = words
5
6     def f(self, pref, suff):
7         # Check from last index to first
8         for i in range(len(self.words) - 1, -1, -1):
9
10            if self.words[i].startswith(pref) and \
11                self.words[i].endswith(suff):
12                return i
13
14        return -1
15
16
17 # Example
18 wordFilter = WordFilter(["apple"])
19
20 print(wordFilter.f("a", "e"))
```

0

=== Code Execution Successful ===

Time Complexity

For each search:

$O(n \times L)$

Where:

- n = number of words
- L = maximum word length

Space Complexity

$O(n)$

Result

Thus, the **WordFilter** class successfully searches for a word using a given prefix and suffix and returns the **largest matching index**.

Experiment:12- Floyd–Warshall Algorithm for All-Pairs Shortest Path

Aim

To implement Floyd's Algorithm to find the shortest distance between all pairs of cities, display the distance matrix before and after applying the algorithm, and find the city with the smallest number of neighbors within the given distance threshold.

Code:

```
1 INF = float('inf')
2
3 def floyd_warshall(n, edges, distanceThreshold):
4     # Initialize distance matrix
5     dist = [[INF] * n for _ in range(n)]
6
7     for i in range(n):
8         dist[i][i] = 0
9
10    # Add edges (undirected Graphs)
11    for u, v, weight in edges:
12        dist[u][v] = weight
13        dist[v][u] = weight
14
15    # Display matrix before Floyd-Warshall
16    print("\nDistance Matrix Before Floyd-Warshall:")
17    for row in dist:
18        print(row)
19
20    # Floyd-Warshall Algorithm
21    for k in range(n):
22        for i in range(n):
23            for j in range(n):
24                if dist[i][k] + dist[k][j] < dist[i][j]:
25                    dist[i][j] = dist[i][k] + dist[k][j]
26
27    # Display matrix after Floyd-Warshall
28    print("\nDistance Matrix After Floyd-Warshall:")
29    for row in dist:
30        print(row)
31
32    # Find city with minimum reachable neighbors
33    min_neighbors = n
34    answer = -1
35
36    for i in range(n):
37        count = 0
38
39        for j in range(n):
40            if i != j and dist[i][j] <= distanceThreshold:
41                count += 1
42
43        print(f"City {i} has {count} neighboring cities.")
44
45    # We ensure greatest city number is selected during tie
46    if count <= min_neighbors:
47        min_neighbors = count
48        answer = i
49
50    return answer
51
52 # Input
53 n = 4
54
55 edges = [
56     [0, 1, 3],
57     [1, 2, 4],
58     [2, 3, 1]
59 ]
60
61 distanceThreshold = 4
62
63 # Find answer
64 result = floyd_warshall(n, edges, distanceThreshold)
65
66 print("\nCity with smallest number of neighbors:", result)
```

Output

```
Distance Matrix Before Floyd-Warshall:
[0, 3, inf, inf]
[3, 0, 4, inf]
[inf, 4, 0, 1]
[inf, 1, 0, 0]

Distance Matrix After Floyd-Warshall:
[0, 3, 4, 5]
[3, 0, 4, 3]
[4, 1, 0, 1]
[0, 2, 1, 0]

City 0 has 3 neighboring cities.
City 1 has 3 neighboring cities.
City 2 has 3 neighboring cities.
City 3 has 2 neighboring cities.

City with smallest number of neighbors: 3
=== Code Execution Successful ===
```

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

Result

Thus, the Floyd–Warshall Algorithm was successfully implemented to find the shortest paths between all pairs of cities. The city with the smallest number of neighbors within the distance threshold 4 is City 3.

Experiment:13- Floyd's Algorithm for Router Network Link Failure

Aim

To implement Floyd's Algorithm to calculate the shortest paths between all pairs of routers and update the network when the link between Router B and Router D fails.

Code

```
17 1
18
19
20 def create_matrix(edges):
21     dist = [[inf] * n for _ in range(n)]
22
23     for i in range(n):
24         dist[i][i] = 0
25
26     for u, v, weight in edges:
27         i = routers.index(u)
28         j = routers.index(v)
29         dist[i][j] = weight
30         dist[j][i] = weight
31
32     return dist
33
34
35
36 def floyd_warshall(dist):
37     dist = [row[:] for row in dist]
38
39     for k in range(n):
40         for i in range(n):
41             for j in range(n):
42                 if dist[i][k] + dist[k][j] < dist[i][j]:
43                     dist[i][j] = dist[i][k] + dist[k][j]
44
45     return dist
46
47
48 def print_matrix(dist):
49     print("\n")
50     for router in routers:
51         print(f"{router}>:", end="")
52         print()
53
54     for i in range(n):
55         print(f"{routers[i]}>:", end="")
56         for j in range(n):
57             if dist[i][j] == inf:
58                 print(f"INF", end="")
59             else:
60                 print(f"{dist[i][j]}>", end="")
61         print()
62
63
64 # Before link failure
65 initial_matrix = create_matrix(edges)
66 print("Distance Matrix Before Floyd's Algorithm:")
67 print_matrix(initial_matrix)
68
69 shortest_before = floyd_warshall(initial_matrix)
70
71 print("\nShortest Distance Matrix Before Link Failure:")
72 print_matrix(shortest_before)
73
74 print("\nRouter A to Router F = ", shortest_before[0][5])
75
76
77 # Remove link B-D
78 updated_edges = []
79
80 for edge in edges:
81     if not ((edge[0] == 'B' and edge[1] == 'D') or
82           (edge[0] == 'D' and edge[1] == 'B')):
83         updated_edges.append(edge)
84
85
86 # After link failure
87 updated_matrix = create_matrix(updated_edges)
88
89 print("\nLink Between Router B and Router D FAILED")
90 print("\nDistance Matrix After Link Failure:")
91 print_matrix(updated_matrix)
92
93 shortest_after = floyd_warshall(updated_matrix)
94
95 print("\nShortest Distance Matrix After Link Failure:")
96 print_matrix(shortest_after)
97
98 print("\nRouter A to Router F = ", shortest_after[0][5])
99
100
```

Output

```
Distance Matrix Before Floyd-Warshall:
[0, 3, inf, inf]
[3, 0, 1, 4]
[inf, 1, 0, 1]
[inf, 4, 1, 0]

Distance Matrix After Floyd-Warshall:
[0, 3, 4, 3]
[3, 0, 1, 2]
[4, 1, 0, 1]
[3, 2, 1, 0]

City 0 has 2 neighboring cities.
City 1 has 3 neighboring cities.
City 2 has 3 neighboring cities.
City 3 has 2 neighboring cities.
City with smallest number of neighbors: 3

*** Code Execution Successful ***
```

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

Result

Thus, Floyd's Algorithm is used to calculate shortest paths between all routers. When the link between **Router B and Router D fails**, the distance matrix is updated and the shortest path from **Router A to Router F** is recalculated.

Experiment:14- Floyd–Warshall Algorithm for All-Pairs Shortest Path

Aim

To implement Floyd's Algorithm to find the shortest paths between all pairs of cities, display the distance matrix before and after applying the algorithm, and identify the city having the minimum number of neighboring cities within the given distance threshold.

Code

```
1 INF = float('inf')
2
3
4 def floyd_warshall(n, edges, distanceThreshold):
5     # Initialize distance matrix
6     dist = [[INF] * n for _ in range(n)]
7
8     # Distance from a city to itself is 0
9     for i in range(n):
10        dist[i][i] = 0
11
12    # Add edges to the distance matrix
13    for u, v, weight in edges:
14        dist[u][v] = weight
15        dist[v][u] = weight
16
17    # Display matrix before applying Floyd's algorithm
18    print("Distance Matrix Before Floyd's Algorithm:")
19    for row in dist:
20        print(row)
21
22    # Floyd-Warshall Algorithm
23    for k in range(n):
24        for i in range(n):
25            for j in range(n):
26                if dist[i][k] + dist[k][j] < dist[i][j]:
27                    dist[i][j] = dist[i][k] + dist[k][j]
28
29    # Display matrix after applying Floyd's algorithm
30    print("\nDistance Matrix After Floyd's Algorithm:")
31    for row in dist:
32        print(row)
33
34    # Find city with minimum neighbors within threshold
35    min_neighbors = float('inf')
36    answer = -1
37
38    for i in range(n):
39        count = 0
40
41        for j in range(n):
42            if i != j and dist[i][j] <= distanceThreshold:
43                count += 1
44
45        print(f"City {i} has {count} neighboring cities.")
46
47        # If the above city has greater number
48        if count <= min_neighbors:
49            min_neighbors = count
50            answer = i
51
52    return answer
53
54
55 # Input
56 n = 5
57
58 edges = [
59     [0, 1, 2],
60     [0, 4, 3],
61     [1, 2, 3],
62     [1, 4, 2],
63     [2, 3, 1],
64     [3, 4, 1]
65 ]
66
67 distanceThreshold = 2
68
69 # Find result
70 result = floyd_warshall(n, edges, distanceThreshold)
71
72 print("\nCity with minimum neighboring cities:", result)
```

```
Distance Matrix Before Floyd's Algorithm:
[0, 2, inf, inf, 3]
[2, 0, 3, inf, 2]
[inf, 3, 0, 1, inf]
[inf, inf, 1, 0, 1]
[3, 2, inf, 1, 0]

Distance Matrix After Floyd's Algorithm:
[0, 2, 5, 3, 4]
[2, 0, 3, 2, 2]
[3, 0, 1, 2]
[3, 1, 0, 1]
[4, 2, 2, 1, 0]

City 0 has 1 neighboring cities.
City 1 has 2 neighboring cities.
City 2 has 2 neighboring cities.
City 3 has 2 neighboring cities.
City 4 has 3 neighboring cities.
City with minimum neighboring cities: 0
*** Code Execution Successful ***
```

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

Result

Thus, the **Floyd–Warshall Algorithm** successfully finds the shortest paths between all pairs of cities. For a distance threshold of **2**, **City 0** has the minimum number of neighboring cities, so the output is **0**.

Experiment:15- Optimal Binary Search Tree (OBST) Using Dynamic Programming

Aim

To construct an Optimal Binary Search Tree for the given keys and frequencies and find the minimum search cost using Dynamic Programming.

Code

```
main.py Visualize Run Output
1 keys = ['A', 'B', 'C', 'D']
2 freq = [0.1, 0.2, 0.3, 0.4]
3
4 n = len(keys)
5
6 # Cost and root matrices
7 cost = [[0.0 for _ in range(n)] for _ in range(n)]
8 root = [[0 for _ in range(n)] for _ in range(n)]
9
10 # Initialize diagonal
11 for i in range(n):
12     cost[i][i] = freq[i]
13     root[i][i] = i
14
15 # Length of subtree
16 for length in range(2, n + 1):
17     for i in range(n - length + 1):
18         j = i + length - 1
19
20         cost[i][j] = float('inf')
21
22         # Sum of frequencies
23         total_freq = sum(freq[i:j + 1])
24
25         # Try each key as root
26         for r in range(i, j + 1):
27
28             left_cost = cost[i][r - 1] if r > i else 0
29             right_cost = cost[r + 1][j] if r < j else 0
30
31             current_cost = left_cost + right_cost + total_freq
32
33             if current_cost < cost[i][j]:
34                 cost[i][j] = current_cost
35                 root[i][j] = r
36
37 # Print cost matrix
38 print("Cost Matrix:")
39 for row in cost:
40     print(row)
41
42 # Print root matrix
43 print("\nRoot Matrix:")
44 for row in root:
45     print(row)
46
47 # Minimum cost
48 print("\nOptimal Cost:", cost[0][n - 1])
49
50
51 # Function to display OBST
52 def print_tree(i, j, parent=None, relation="Root"):
53     if i > j:
54         return
55
56     r = root[i][j]
57
58     if parent is None:
59         print(keys[r], "is the Root")
60     else:
61         print(keys[r], "is", relation, "child of", keys[parent])
62
63     print_tree(i, r - 1, r, "left")
64     print_tree(r + 1, j, r, "right")
65
66 print("\nOptimal Binary Search Tree:")
67 print_tree(0, n - 1)
```

```
Cost Matrix:
[0.1, 0.4, 1.0, 1.8]
[0.0, 0.2, 0.7, 1.5]
[0.0, 0.0, 0.3, 1.0]
[0.0, 0.0, 0.0, 0.4]

Root Matrix:
[0, 1, 1, 2]
[0, 1, 2, 2]
[0, 0, 2, 2]
[0, 0, 0, 3]

Optimal Cost: 1.7

Optimal Binary Search Tree:
C is the Root
B is left child of C
A is left child of B
D is right child of C
```

*** Code Execution Successful ***

Time Complexity

$O(N^3)$

Space Complexity

$O(N^2)$

Result

Thus, the Optimal Binary Search Tree was successfully constructed using Dynamic Programming, and the minimum search cost obtained is:

Optimal Cost = 1.7

Experiment:16- Optimal Binary Search Tree (OBST) Using Dynamic Programming

Aim

To construct an Optimal Binary Search Tree (OBST) for the given keys and frequencies and find its minimum search cost and root matrix.

Code

```
main.py
1 keys = [10, 12, 16, 21]
2 freq = [4, 2, 0, 3]
3
4 n = len(keys)
5
6 # cost[i][j] stores minimum cost of OBST from i to j
7 cost = [[0] * n for _ in range(n)]
8
9 # root[i][j] stores index of root
10 root = [[0] * n for _ in range(n)]
11
12 # Initialize single keys
13 for i in range(n):
14     cost[i][i] = freq[i]
15     root[i][i] = i
16
17 # Build OBST for different subtree lengths
18 for length in range(2, n + 1):
19     for i in range(n - length + 1):
20         j = i + length - 1
21
22         cost[i][j] = float('inf')
23         total_freq = sum(freq[i:j + 1])
24
25         # Try every key as root
26         for r in range(i, j + 1):
27
28             left_cost = cost[i][r - 1] if r > i else 0
29             right_cost = cost[r + 1][j] if r < j else 0
30
31             current_cost = left_cost + right_cost + total_freq
32
33             if current_cost < cost[i][j]:
34                 cost[i][j] = current_cost
35                 root[i][j] = r
36
37 # Print Cost Matrix
38 print("Cost Matrix:")
39 for row in cost:
40     print(row)
41
42 # Print Root Matrix
43 print("\nRoot Matrix:")
44 for row in root:
45     print(row)
46
47 # Print minimum cost
48 print("\nOptimal Cost =", cost[0][n - 1])
49
50 # Function to print OBST
51 def print_tree(i, j, parent=None, relation="Root"):
52     if i > j:
53         return
54
55     r = root[i][j]
56
57     if parent is None:
58         print(keys[r], "is the Root")
59     else:
60         print(keys[r], "is the", relation,
61               "child of", keys[parent])
62
63     print_tree(i, r - 1, r, "left")
64     print_tree(r + 1, j, r, "right")
65
66 print("\nOptimal Binary Search Tree:")
67 print_tree(0, n - 1)
```

Output

```
Cost Matrix:
[4, 6, 20, 26]
[0, 2, 10, 16]
[0, 0, 0, 12]
[0, 0, 0, 3]

Root Matrix:
[0, 0, 2, 2]
[0, 1, 2, 2]
[0, 0, 2, 2]
[0, 0, 0, 3]

Optimal Cost = 26

Optimal Binary Search Tree:
16 is the Root
10 is the left child of 16
12 is the right child of 10
21 is the right child of 12

*** Code Execution Successful ***
```

Time Complexity

$O(N^3)$

Space Complexity

$O(N^2)$

Result

Thus, the Optimal Binary Search Tree was successfully constructed using Dynamic Programming, and the minimum cost obtained is:

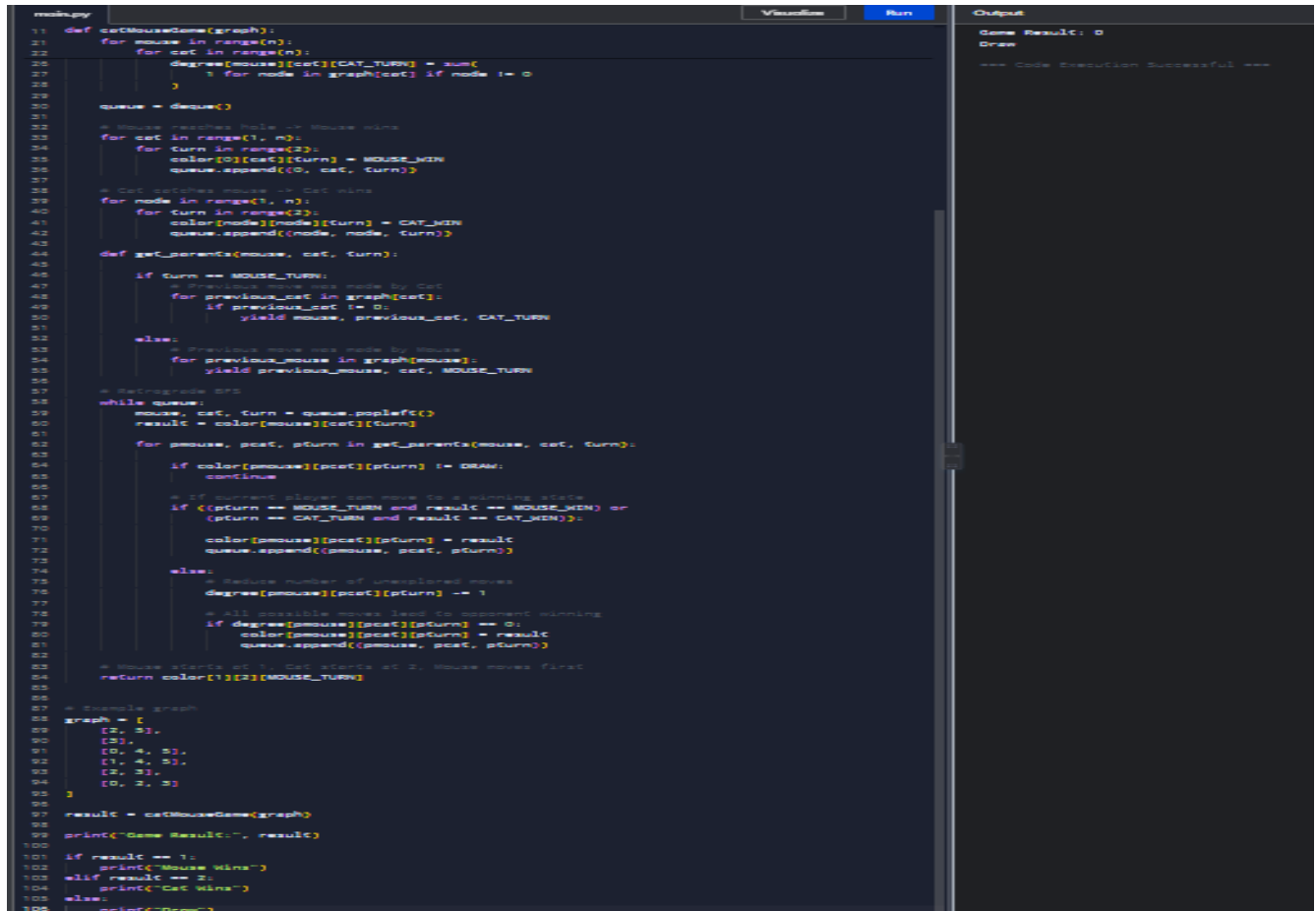
Optimal Cost = 26

Experiment:17- Cat and Mouse Game Using Graph Algorithm

Aim

To write a Python program to determine whether the Mouse wins (1), Cat wins (2), or the game results in a Draw (0) when both players play optimally.

Code



```
main.py
def catMouseGame(graph):
    for mouse in range(n):
        for cat in range(n):
            degree[mouse][cat][CAT_TURN] = sum(
                1 for node in graph[cat] if node != 0
            )
            queue = deque()
            # Mouse reaches hole or Mouse wins
            for turn in range(2):
                for cat in range(2):
                    color[0][cat][turn] = MOUSE_WIN
                    queue.append((0, cat, turn))
            # Cat catches mouse or Cat wins
            for node in range(1, n):
                for turn in range(2):
                    color[node][node][turn] = CAT_WIN
                    queue.append((node, node, turn))
            def get_parents(mouse, cat, turn):
                if turn == MOUSE_TURN:
                    # Previous move was made by cat
                    for previous_cat in graph[cat]:
                        if previous_cat != 0:
                            yield mouse, previous_cat, CAT_TURN
                else:
                    # Previous move was made by Mouse
                    for previous_mouse in graph[mouse]:
                        yield previous_mouse, cat, MOUSE_TURN
            # Breadth-first search
            while queue:
                mouse, cat, turn = queue.popleft()
                result = color[mouse][cat][turn]
                for pmouse, pcat, pturn in get_parents(mouse, cat, turn):
                    if color[pmouse][pcat][pturn] != DRAW:
                        continue
                    # If current player can move to a winning state
                    if ((pturn == MOUSE_TURN and result == MOUSE_WIN) or
                        (pturn == CAT_TURN and result == CAT_WIN)):
                        color[pmouse][pcat][pturn] = result
                        queue.append((pmouse, pcat, pturn))
                    else:
                        # Reduce number of unvisited moves
                        degree[pmouse][pcat][pturn] -= 1
                        # All possible moves lead to current winning
                        if degree[pmouse][pcat][pturn] == 0:
                            color[pmouse][pcat][pturn] = result
                            queue.append((pmouse, pcat, pturn))
            # Mouse starts at 1, Cat starts at 2, Mouse moves first
            return color[1][2][MOUSE_TURN]

# Example graph
graph = [
    [2, 3],
    [2],
    [0, 4, 5],
    [1, 4, 5],
    [0, 3],
    [0, 2, 5]
]

result = catMouseGame(graph)
print("Game Result:", result)

if result == 1:
    print("Mouse Wins")
elif result == 2:
    print("Cat Wins")
else:
    print("Draw")
```

Game Result: 0
Draw
--- Code Execution Successful ---

Time Complexity

$O(n^2)$

Space Complexity

$O(n^2)$

Result

Thus, the Cat and Mouse game is solved using graph-state analysis. The program determines the outcome assuming both the Mouse and Cat play optimally.

Experiment:18- Path with Maximum Probability Using Dijkstra's Algorithm

Aim

To write a Python program to find the path between two nodes with the maximum probability of successful traversal.

Code

```
main.py Visualize Run Output
1 import heapq
2
3
4 def maxProbability(n, edges, succProb, start, end):
5     # Create adjacency list
6     graph = [[] for _ in range(n)]
7     for (u, v), probability in zip(edges, succProb):
8         graph[u].append((v, probability))
9         graph[v].append((u, probability))
10
11     # Maximum probability for each node
12     probability = [0.0] * n
13     probability[start] = 1.0
14
15     # Max heap using negative probability
16     heap = [(-1.0, start)]
17
18     while heap:
19         current_prob, node = heapq.heappop(heap)
20         current_prob = -current_prob
21
22         # Destination reached
23         if node == end:
24             return current_prob
25
26         # Skip outdated entries
27         if current_prob < probability[node]:
28             continue
29
30         # Check neighboring nodes
31         for neighbor, edge_prob in graph[node]:
32             new_prob = current_prob * edge_prob
33
34             if new_prob > probability[neighbor]:
35                 probability[neighbor] = new_prob
36                 heapq.heappush(heap, (-new_prob, neighbor))
37
38     return 0.0
39
40
41 # Input
42 n = 3
43 edges = [(0, 1), (1, 2), (0, 2)]
44 succProb = [0.5, 0.5, 0.2]
45 start = 0
46 end = 2
47
48 # Find maximum probability
49 result = maxProbability(n, edges, succProb, start, end)
50
51 print("Maximum Probability:", result)
```

Maximum Probability: 0.25

=== Code Execution Successful ===

Time Complexity

$O((V + E) \log V)$

Space Complexity

$O(V + E)$

Result

Thus, the program successfully finds the path with the maximum probability of success using a modified Dijkstra's Algorithm.

Experiment:19- Unique Paths in a Grid Using Dynamic Programming

Aim

To write a Python program to find the number of unique paths for a robot to move from the top-left corner to the bottom-right corner of an $m \times n$ grid, where the robot can move only Right or Down.

Code

main.py	Visualization	Run	Output
<pre>1 def uniquePaths(m, n): 2 3 # Create DP table 4 dp = [[0] * n for _ in range(m)] 5 6 # There is only one way to reach 7 # cells in the first row and first column 8 for i in range(m): 9 dp[i][0] = 1 10 11 for j in range(n): 12 dp[0][j] = 1 13 14 # Calculate number of paths 15 for i in range(1, m): 16 for j in range(1, n): 17 dp[i][j] = dp[i-1][j] + dp[i][j-1] 18 19 return dp[m-1][n-1] 20 21 22 # Test Case 1 23 m = 3 24 n = 7 25 26 print("Number of Unique Paths:", uniquePaths(m, n))</pre>			<pre>Number of Unique Paths: 28 *** Code Execution Successful ***</pre>

Time Complexity

$O(m \times n)$

Space Complexity

$O(m \times n)$

Result

Thus, the program successfully calculates the number of unique paths using Dynamic Programming. The robot can reach each cell either from the top or from the left, so the number of paths is calculated by adding these two possibilities.

Experiment:20- Number of Good Pairs

Aim

To write a Python program to find the number of good pairs (i, j) such that:

- $\text{nums}[i] == \text{nums}[j]$
- $i < j$

Code

<pre>main.py Validate Run</pre> <pre>1 def numIdenticalPairs(nums): 2 count = {} 3 good_pairs = 0 4 5 for num in nums: 6 # Add the number of previous occurrences 7 good_pairs += count.get(num, 0) 8 9 # Update frequency 10 count[num] = count.get(num, 0) + 1 11 12 return good_pairs 13 14 15 # Test Case 1 16 nums = [1, 2, 3, 1, 1, 3] 17 18 print("Number of Good Pairs:", numIdenticalPairs(nums))</pre>	<pre>Output</pre> <pre>Number of Good Pairs: 4 === Code Execution Successful ===</pre>
--	--

Time Complexity

$O(n)$

Space Complexity

$O(n)$

Result

Thus, the program successfully finds the total number of **good pairs** in the given array.

Experiment:21- Find the City with the Smallest Number of Reachable Cities

Aim

To write a Python program to find the city that has the smallest number of cities reachable within a given distance threshold. If there is a tie, return the city with the greatest index.

Code

```
realnipy
def findTheCity(n, edges, distanceThreshold):
    INF = float('inf')

    # Create distance matrix
    dist = [[INF] * n for _ in range(n)]

    # Distance from a city to itself is 0
    for i in range(n):
        dist[i][i] = 0

    # Add bidirectional edges
    for u, v, weight in edges:
        dist[u][v] = weight
        dist[v][u] = weight

    # Floyd-Warshall Algorithm
    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(
                    dist[i][j],
                    dist[i][k] + dist[k][j]
                )

    # Find city with minimum reachable cities
    minimum_count = float('inf')
    answer = -1

    for i in range(n):
        count = 0

        for j in range(n):
            if i != j and dist[i][j] <= distanceThreshold:
                count += 1

        # ensures greater city index is selected in a tie
        if count <= minimum_count:
            minimum_count = count
            answer = i

    return answer

# Example Input
n = 4
edges = [
    [0, 1, 3],
    [1, 2, 1],
    [1, 3, 4],
    [2, 3, 1]
]
distanceThreshold = 4
result = findTheCity(n, edges, distanceThreshold)
print("City with minimum reachable cities:", result)
```

City with minimum reachable cities: 3
=== Code Execution Successful ===

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

Result

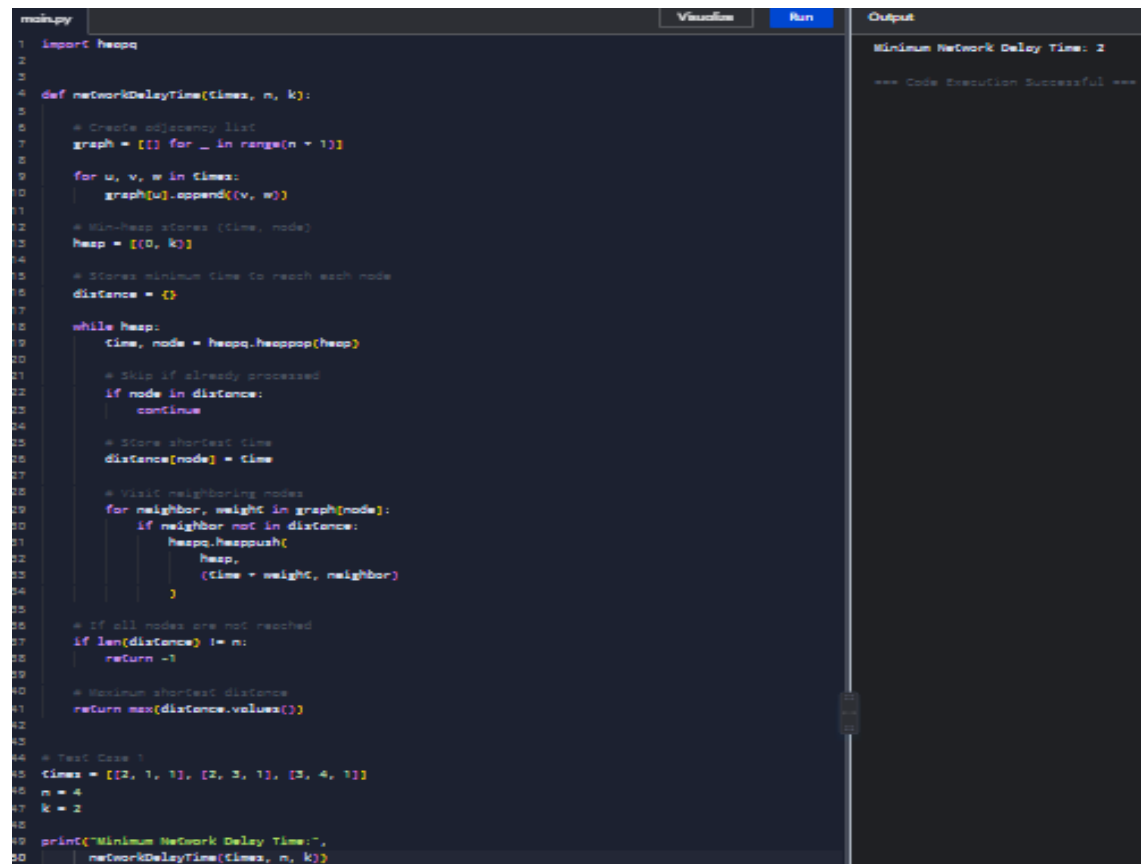
Thus, the program successfully finds the city with the **smallest number of reachable cities within the given distance threshold** using the **Floyd–Warshall Algorithm**.

Experiment:22- Network Delay Time Using Dijkstra's Algorithm

Aim

To write a Python program to find the minimum time required for a signal sent from node k to reach all nodes in a directed network.

Code



```
main.py Viewline Run Output
1 import heapq
2
3
4 def networkDelayTime(times, n, k):
5     # Create adjacency list
6     graph = [[] for _ in range(n + 1)]
7     for u, v, w in times:
8         graph[u].append((v, w))
9
10    # Min-heap stores (time, node)
11    heap = [(0, k)]
12
13    # Stores minimum time to reach each node
14    distance = {}
15
16    while heap:
17        time, node = heapq.heappop(heap)
18
19        # Skip if already processed
20        if node in distance:
21            continue
22
23        # Store shortest time
24        distance[node] = time
25
26        # Visit neighboring nodes
27        for neighbor, weight in graph[node]:
28            if neighbor not in distance:
29                heapq.heappush(
30                    heap,
31                    (time + weight, neighbor)
32                )
33
34    # If all nodes are not reached
35    if len(distance) != n:
36        return -1
37
38    # Maximum shortest distance
39    return max(distance.values())
40
41
42
43
44 # Test Case 1
45 times = [[2, 1, 1], [2, 3, 1], [3, 4, 1]]
46 n = 4
47 k = 2
48
49 print("Minimum Network Delay Time:",
50       networkDelayTime(times, n, k))
```

Minimum Network Delay Time: 2
*** Code Execution Successful ***

Time Complexity

$O((V + E) \log V)$

Where:

- V = Number of nodes
- E = Number of edges

Space Complexity

$O(V + E)$

Result

Thus, the program successfully uses **Dijkstra's Algorithm** to calculate the minimum time required for a signal to reach all nodes. If any node cannot be reached, the program returns **-1**.

